



Preference learning based deep reinforcement learning for flexible job shop scheduling problem

Xinning Liu¹ · Li Han¹ · Ling Kang² · Jiannan Liu¹ · Huadong Miao³

Received: 6 June 2024 / Accepted: 29 December 2024 / Published online: 15 January 2025
© The Author(s) 2025

Abstract

The flexible job shop scheduling problem (FJSP) holds significant importance in both theoretical research and practical applications. Given the complexity and diversity of FJSP, improving the generalization and quality of scheduling methods has become a hot topic of interest in both industry and academia. To address this, this paper proposes a Preference-Based Mask-PPO (PBMP) algorithm, which leverages the strengths of preference learning and invalid action masking to optimize FJSP solutions. First, a reward predictor based on preference learning is designed to model reward prediction by comparing random fragments, eliminating the need for complex reward function design. Second, a novel intelligent switching mechanism is introduced, where proximal policy optimization (PPO) is employed to enhance exploration during sampling, and masked proximal policy optimization (Mask-PPO) refines the action space during training, significantly improving efficiency and solution quality. Furthermore, the Pearson correlation coefficient (PCC) is used to evaluate the performance of the preference model. Finally, comparative experiments on FJSP benchmark instances of varying sizes demonstrate that PBMP outperforms traditional scheduling strategies such as dispatching rules, OR-Tools, and other deep reinforcement learning (DRL) algorithms, achieving superior scheduling policies and faster convergence. Even with increasing instance sizes, preference learning proves to be an effective reward mechanism in reinforcement learning for FJSP. The ablation study further highlights the advantages of each key component in the PBMP algorithm across performance metrics.

Keywords Flexible job shop scheduling problem · Preference learning · Proximal policy optimization · Deep reinforcement learning

Introduction

With the rapid advancement of technologies such as the Internet of Things (IoT), big data, and artificial intelligence (AI), modern manufacturing is undergoing significant transformation, driving the adoption of intelligent manufacturing systems. In the face of increasing product differentiation and customization issues in manufacturing processes, efficient production scheduling algorithms are needed to quickly adapt to unknown changes in the production environment or

manufacturing systems, thereby improving factory productivity and optimizing resource utilization in real time [1].

At the same time, the increasing complexity and customization demands in manufacturing have led to a growing need for efficient and flexible production scheduling algorithms. These algorithms play a critical role in enhancing factory productivity and resource optimization, especially in highly dynamic and uncertain production environments.

The flexible job shop scheduling problem (FJSP) is an extension of the classic job shop scheduling problem, introducing flexibility in machine selection and operation sequencing, making it more representative of real-world production scenarios. However, this flexibility also significantly increases the complexity of the problem. FJSP is classified as an NP-hard problem, with its computational complexity growing exponentially as the problem size increases [2, 3]. It expands the search space of feasible solutions and becomes a more complex NP-hard problem than the JSP [4, 5]. This complexity has made FJSP a key focus of research in both

✉ Xinning Liu
liuxinning@neusoft.edu.cn

¹ School of Computer and Software, Dalian Neusoft University of Information, Dalian 116023, Liaoning, China

² Neusoft Research Institute, Dalian Neusoft University of Information, Dalian 116023, Liaoning, China

³ SNOW China (Beijing) Co. Ltd., Dalian Branch, Dalian 116023, Liaoning, China

academia and industry, particularly in highly customized sectors such as automotive manufacturing, electronics, and aerospace.

Efficiently solving FJSP is of great importance because it has widespread applications in various industrial settings. In modern factories, production systems frequently encounter dynamic changes such as machine breakdowns, task reassignment, and random job insertions. These challenges require scheduling solutions that can dynamically adjust to changing conditions while minimizing production delays and optimizing resource allocation. Successfully solving FJSP not only enhances production flexibility but also reduces operational costs and lead times, which are essential for maintaining competitiveness in today's global market.

Furthermore, FJSP often involves multi-objective optimization in practical applications, where objectives such as minimizing makespan, reducing energy consumption, and improving production efficiency must be considered simultaneously. Traditional methods, such as heuristic and meta-heuristic algorithms, although capable of providing reasonable solutions in some cases, often fall short in dynamic and multi-objective scenarios. These methods tend to struggle with finding globally optimal solutions and are prone to local optima, especially when dealing with large-scale and unpredictable environments.

In recent years, deep reinforcement learning (DRL) has emerged as a promising approach for addressing the complexities of FJSP. DRL integrates the decision-making capabilities of reinforcement learning with the representational power of deep learning, allowing it to autonomously learn and optimize complex scheduling strategies through interaction with dynamic environments. This makes DRL particularly suitable for real-time scheduling in highly uncertain and variable production settings. Despite these advancements, applying DRL to FJSP still presents several challenges. One major obstacle is the design of effective reward functions that accurately reflect the quality of a scheduling strategy. Additionally, DRL systems often face difficulties in balancing exploration and exploitation, especially in large action spaces, and are prone to selecting invalid actions during training, which can degrade overall performance.

Therefore, this paper proposes a preference-based mask-PPO (PBMP) algorithm framework to address these challenges in FJSP, and PBMP makes the following contributions:

- (1) Preference learning for FJSP: For the first time, preference learning is applied to the flexible job shop scheduling problem (FJSP). It demonstrates superior generalization across FJSP instances of varying sizes, particularly excelling in large-scale instances. Extensive experiments show that it consistently outperforms

dispatching rules and traditional DRL algorithms, delivering highly effective scheduling policies. The ablation study further highlights the critical role of each key component in the preference learning, demonstrating their significant contribution to the overall performance.

- (2) Intelligent switching in DRL: A novel intelligent switching mechanism is established to enhance DRL during sampling and training. PPO is used in the sampling phase to improve sample diversity, while a random policy enhances exploration. During training, the algorithm switches to Mask-PPO to reduce the action space, improving training efficiency and solution quality.
- (3) Improved preference model: Clipping and normalization techniques are firstly introduced to optimize the Bradley–Terry model, effectively preventing overflow issues during backpropagation. These innovations lead to more stable training and enhanced model performance, representing a significant advancement in preference learning for FJSP.
- (4) Performance evaluation using PCC: The Pearson correlation coefficient (PCC) is introduced as a novel new metric to evaluate the preference. PCC effectively measures how well the reward values generated by the preference model align with the actual makespan, offering a more accurate assessment of the model's ability to capture the complex relationship between the reward and makespan. It demonstrates that the preference model consistently maintains a strong negative correlation across FJSP instances of varying sizes.

These contributions collectively advance the state-of-the-art in FJSP solutions and have significant practical implications for industries that require adaptive and flexible scheduling systems. The proposed framework can be effectively applied to a wide range of industrial settings, from small manufacturing workshops to large-scale production environments, offering enhanced efficiency, reduced costs, and increased competitiveness.

The remainder of this paper is organized as follows: The "Literature review" section offers an extensive review of existing studies, highlighting the gaps in current research within the field. "The FJSP and preliminaries" establishes the mathematical model for FJSP and introduces the key components of the Mask-PPO and preference learning algorithms. "Proposed methods for FJSP" presents the PBMP algorithm and its application to FJSP. In "Experiment and results", we conduct numerical experiments and analyze the results. Finally, "Conclusions and future work" concludes the paper with insights and future research directions.

Literature review

Methods for scheduling

In this section, we review the literature related to FJSP. The literature review indicates that there are three main types of solutions for FJSP problems: heuristic rules, meta-heuristic algorithms, and reinforcement learning. Panwalkar et al. [6] provided an overview of various scheduling rules and their effectiveness in different manufacturing environments. Pezzella et al. [7] introduced a genetic algorithm for the flexible job-shop scheduling problem (FJSP), demonstrating its ability to improve solution quality and computational efficiency. Sutton et al. [8] offered a comprehensive introduction to reinforcement learning, laying the groundwork for its application in dynamic decision-making problems like scheduling. For automobile metal parts manufacturing plants, Calleja et al. [9] proposed a scheduling rule algorithm to solve the FJSP of load transfer batches, and also provided a variant model for manually assigning weights to priority rules. Ortíz et al. [10] studied a scheduling algorithm and model for flexible workshop systems in the field of textiles with quality constraints. Saidi-mehrabad et al. [11] proposed a tabu search algorithm consisting of two parts. The algorithm can produce optimal solutions for small to medium-sized problems in a shorter computational time. Nouri et al. [12] applied the particle swarm optimization algorithm to solve the two sub-problems of machine selection and process sequencing. Braune et al. [13] used a genetic algorithm to generate priority rules for job scheduling in the form of an expression tree. Heuristic algorithms exhibit poor global performance, and the gap between their solutions and the optimal solution is difficult to estimate. Moreover, due to their short-sighted nature, it is challenging to ensure consistent scheduling performance across different environments. Meta-heuristic algorithms have high computational complexity and are prone to falling into local optima. To ensure both efficiency and quality, it is necessary to automatically construct scheduling solutions at different decision steps based on the current scheduling state.

In recent years, reinforcement learning (RL) has demonstrated promising results in improving the efficiency of scheduling algorithms. The following review will primarily focus on reinforcement learning methods, which is the main direction we utilize and investigate in this paper. RL's adaptability to dynamic and uncertain environments allows systems to learn scheduling strategies through experience. Zhang et al. [14] applied reinforcement learning to optimize static job-shop scheduling, improving job allocation efficiency. Xue et al. [15] used Q-learning to address multi-AGV scheduling in manufacturing, enhancing the coordination of AGVs in workshops. Building on the foundations of

RL, deep reinforcement learning (DRL) has gained significant attention for combining the representational power of deep learning with the decision-making capabilities of RL. Similarly, Shi et al. [16] used DRL to address production line conflicts, while Han et al. [17] enhanced DRL with a three-dimensional disjunctive graph to better capture the complexity of flexible production conditions. Oh et al. [18] combined independent learners with the implicit quantile network and built a multi-agent scheduling algorithm that exhibits superior performance. Zhang et al. [19] integrated DRL with MARL by associating different agents with each machine and job, building a multi-agent graph based on the operational relationships between machines and jobs, and facilitating cooperation among the agents. Du et al. [20] proposed a hybrid multi-objective optimization algorithm using the estimation of distribution algorithm and DQN to solve the flexible job shop scheduling problem with time-of-use electricity price constraints. Bonetta et al. [21] proposed an end-to-end deep reinforcement learning (DRL) approach that eliminates the need for hand-crafted heuristic rules. This method can automatically learn scheduling rules and demonstrates competitive performance when compared with classical methods and the latest DRL techniques. Li et al. [22] combined co-evolution mechanisms with DRL to dynamically adjust job assignments and machine selections, optimizing both energy consumption and production efficiency. Ren et al. [23] optimized scheduling decisions by considering the interactions between human operators and automated agents, using DRL to balance competition and collaboration among the agents, thereby improving scheduling performance. Yuan et al. [24] designed a new DRL framework that extracts state information using a lightweight multi-layer perceptron, with newly designed state representations and action spaces, improving optimization ability and applicability. Xu et al. [25] transformed the FJSP into a Markov decision process, using a transformer encoder for feature extraction, and designed composite dispatching rules and a reward function to achieve flexible and efficient scheduling optimization through DRL. Luo et al. [26] addressed the dynamic multi-objective flexible job shop scheduling problem (DMOFJSP), proposing an on-line rescheduling framework named as two-hierarchy deep Q network (THDQN).

Proximal policy optimization (PPO), a policy-based DRL algorithm, has shown superior performance in handling large action spaces in scheduling environments. PPO directly optimizes policies, making it more efficient for solving dynamic FJSP problems. Lei et al. [27] developed a Multi-Proximal Policy Optimization (Multi-PPO) algorithm, which excelled at learning job operation policies and machine action policies in complex scheduling tasks. Zhao et al. [28] introduced a reactive scheduling approach using proximal policy

optimization with an attention-based policy network (PPO-APN). This approach enabled real-time decision-making in dynamic scheduling scenarios. Additionally, a global/local reward function (GLRF) was developed to mitigate the issue of sparse rewards encountered during the training phase. Chang et al. [29] used graph neural networks to embed the disjunctive graph of the FJSP into states for the agent to learn, then trained the model using PPO. They tested the model on various-sized problems, verifying its effectiveness. Hammami et al. [30] formulated the objective function as a weighted sum of the schedule completion time and the deviation from the initially generated schedule. They employed the Proximal Policy Optimization Actor-Critic (PPO-AC) algorithm, combined with an event-driven rescheduling strategy, to address a bi-objective decision problem.

In addition to PPO, researchers have begun exploring hybrid approaches that combine DRL with other optimization techniques. Heger et al. [31] trained a reinforcement learning model that dynamically adjusts sequencing rules for job shop scheduling, providing a human-comprehensible hyper-heuristic. Long et al. [32] proposed an RL-enhanced artificial bee colony algorithm that improved the convergence rate and accuracy of scheduling solutions. Song et al. [33] developed a heterogeneous graph neural network-based architecture integrated with DRL to capture complex relationships between operations and machines, yielding high-quality priority dispatching rules (PDRs). Du et al. [34] introduced a novel framework combining double deep Q-networks (DQN) with a soft target weight update strategy to optimize scheduling tasks, particularly in crane transportation contexts. Yuan et al. [35] employed a multi-agent double DQN algorithm to implement multi-agent reinforcement learning for scheduling, while Yang et al. [36] designed efficient rewards, actions, and state features for both scheduling and reconfiguration agents, building an intelligent scheduling and reconfiguration system based on DRL. Similarly, Jing et al. [37] introduced a multi-agent reinforcement learning (MARL) approach using graph convolutional networks (GCNs) to model job-machine relationships, enabling agents to collaborate effectively in dynamic settings. Huang et al. [38] proposed a lightweight graph gated channel transformation (GGCT) model and improved disjunctive graph feature representation, utilizing a deep reinforcement learning framework to capture key information in FJSP and address inefficiencies in strategies after scaling. Tang et al. [39] combined the heterogeneous graph neural network based on relation (HGNNR) with PPO, using heterogeneous graphs to model the complex relationships in FJSP, and employed the proximal policy optimization (PPO) algorithm to iteratively update policies to maximize cumulative rewards. Zhu et al. [40] proposed a hybrid method combining Q-Learning with meta-heuristics to enhance exploration and exploitation for better scheduling in complex environments.

Based on the literature review above, it is evident that solutions to the FJSP are evolving toward more intelligent and efficient approaches. Research has shifted from traditional heuristic and metaheuristic algorithms to advanced techniques such as DRL, with a particular emphasis on achieving efficient scheduling in complex production environments. As DRL technologies advance, a growing number of studies are incorporating optimization methods, such as dispatching rules, disjunctive graphs, and others, to enhance system scalability and efficiency. Furthermore, PPO-based algorithms, which combine multiple optimization methods, have demonstrated remarkable performance in directly optimizing scheduling strategies. However, these methods still face challenges related to scalability in large-scale scheduling problems and the design of complex reward functions. Addressing these issues remains a critical area for future research. Consequently, the development of efficient and highly scalable scheduling algorithms remains a fundamental goal in FJSP research.

Research gap

Despite significant advancements in DRL and PPO for solving FJSP, several challenges remain unresolved.

Firstly, the reward function design in DRL systems is crucial but often suboptimal. In complex and dynamic environments, it is difficult to design a reward function that accurately reflects the trade-offs between multiple objectives, such as minimizing makespan, energy consumption, and operational costs. Poor reward design can lead to inefficient scheduling strategies, excessive exploration, or premature convergence.

Another critical challenge lies in the definition of state and action spaces. Most current DRL approaches rely on simplified state representations that fail to capture the full complexity of industrial scheduling problems. Additionally, large action spaces increase the likelihood of sampling invalid actions, which negatively impacts the learning process. Although action masking techniques have been proposed to mitigate this issue, these methods are not scalable to larger instances of FJSP.

Scalability is another major research gap. DRL algorithms, particularly PPO, perform well on small to medium-sized FJSP instances but struggle with scalability when applied to large-scale industrial environments. As the number of machines and jobs increases, the computational demands of DRL models grow exponentially, limiting their applicability in real-world scenarios.

Lastly, balancing exploration and exploitation remains a significant challenge. In dynamic production environments, it is essential to explore new strategies without neglecting known optimal solutions. Current DRL systems often lack efficient mechanisms to balance these two aspects, leading to

suboptimal scheduling decisions, especially in environments with frequent disruptions like machine breakdowns or task insertions.

The FJSP and preliminaries

This study focuses on FJSP and proposes algorithms combining Mask-PPO with preference learning to address this problem. Therefore, this section will briefly describe FJSP using mathematical formulas and introduce the principles of the Mask-PPO algorithm and preference learning algorithms.

Problem formulation

The FJSP typically involves n jobs $J_i = (J_1, J_2, \dots, J_n)$, each with one or multiple operations. These operations can be processed on any machine among m machines $M = (M_1, M_2, \dots, M_m)$. Each job is processed along different processing paths according to its requirements, but the sequence of processes must be followed. The essence of scheduling problem research is to determine whether jobs can be processed on certain machines. Depending on the chosen processing machine, the processing time varies. To achieve optimal production, arranging the operations of jobs and selecting appropriate processing machines for the jobs are crucial aspects of the entire scheduling plan.

To obtain the optimal scheduling solution, when dealing with the FJSP, consideration should be given to both the processing sequence of operations and the choice of processing machines. The solution must adhere to the following constraint conditions:

- (1) A machine cannot handle multiple processes simultaneously;
- (2) A job cannot be processed by multiple machines at the same time, and queue cutting is not allowed;
- (3) At time zero, all workpieces have the same probability of being selected;
- (4) The processing steps of a job cannot be adjusted, but the steps of different jobs can be adjusted;
- (5) Different jobs have the same priority.

Table 1 shows the description of relevant variables for FJSP.

Based on the parameter and variable definitions of FJSP described above, and based on the problem description and hypothetical constraints of FJSP, the mathematical model constraints of FJSP can be obtained as formulas (1) to (8), where N is a maximum positive number:

$$C_{ij} \geq S_{ij} + p_{ijk} - (1 - x_{ijk}) \times N, \quad \forall i, j, k \quad (1)$$

Table 1 Description of relevant variables for FJSP

Variable	Description
n	Number of jobs to be processed
m	Number of machines
J_i	The i -th job
O_{ij}	The j -th process of the job
M_k	The k -th machine
n_i	The number of processes for j_i
p_{ijk}	The time required for the j -th process of j_i to be processed on machine m_k
J	A set of jobs, $J_i = \{J_1, J_2, \dots, J_n\}$
M	A set of machines, $M_i = \{M_1, M_2, \dots, M_n\}$
m_{ij}	A set of machines that can be processed, $\mathcal{M}_{ij} \subset \mathcal{M}$
O_i	A set of process for J_i , $O_i = \{O_{i1}, O_{i2}, \dots, O_{in_i}\}$
C_i	$C_i = \max_j C_{ij} = C_{in_i}$
O	A set of process, $O_i = \{O_1, O_2, \dots, O_n\}$
S_{ij}	The start processing time of the process O_{ij}
x_{ijk}	Is the process being processed on the machine M_k ? Yes, it is 1, otherwise it is 0
C_{ij}	The end processing time of the process O_{ij}
C_i	The end processing time of job J_i , $C_i = \max_j C_{ij} = C_{in_i}$
$y_{ji'j'k}$	If the process O_{ij} was processed on the machine M_k before the process $O_{i'j'}$, then it is 1; If the process O_{ij} is processed on machine M_k after the last or two different processes, then it is 0

$$S_{ij} \geq C_{i'j'} - y_{iji'j'k} \times N, \quad \forall i, j, \forall M_k \in \mathcal{M}_{ij} \cap \mathcal{M}_{i'j'} \quad (2)$$

$$S_{i'j'} \geq C_{ij} - (1 - y_{iji'j'k}) \times N, \quad \forall i, j, \forall M_k \in \mathcal{M}_{ij} \cap \mathcal{M}_{i'j'} \quad (3)$$

$$S_{ij} \geq C_{i(j-1)}, \quad \forall i, \forall j = 2, \dots, n_i \quad (4)$$

$$\sum_{k=0}^m x_{ijk} = 1, \quad \forall i, j \quad (5)$$

$$S_{ij} \geq 0, C_{ij} \geq 0, C_i \geq 0, \quad \forall i, j \quad (6)$$

$$x_{ijk} \in \{0, 1\}, \quad \forall i, j, k \quad (7)$$

$$y_{iji'j'k} \in \{0, 1\}, \quad \forall i, j, i', j', \forall M_k \in \mathcal{M}_{ij} \cap \mathcal{M}_{i'j'} \quad (8)$$

The objective function is shown in Eq. (9).

$$C_{\max} = \min(\max_i C_i) \quad (9)$$

The maximum completion time is the maximum final completion time of each workpiece, and the optimization goal is to minimize the maximum completion time.

A common challenge faced by traditional DRL algorithms in FJSP revolves around the design of the reward function. The design of the reward function directly impacts the quality of the scheduling policy. If a reward value is returned at each step, the design of the reward function becomes considerably challenging, as the actual result of the schedule can only be truly assessed upon completion. Conversely, if the reward value is provided when scheduling is complete, while simplifying the design of the reward function, it introduces difficulties in training due to sparse rewards.

Therefore, for the first time, preference learning is used to generate a reward predictor in FJSP. Specifically, trajectory preferences are utilized to evaluate the outcomes of full trajectories.

Preliminaries

Mask-PPO

Mask-PPO is a variant of proximal policy optimization (PPO) that incorporates the concept of masking [41]. The motivation for introducing masking into the PPO framework is driven by the aim to enhance training efficiency and improve the generalization capabilities of the model.

Actions sampled from the full discrete action distribution predicted by the learned policy may be invalid due to the complexity of the rules. Mask-PPO addresses this issue by masking out invalid actions, applying a state-dependent differentiable function during the calculation of the action probability distribution. As a result, it generates a behavior policy that avoids selecting invalid actions. This approach not only guarantees the validity of action selection but also maintains the differentiability of the entire policy update process, allowing the algorithm to continue optimizing using policy gradient methods. The general objective of policy gradient estimation is illustrated in Eq. (10).

$$\nabla_{\theta} J = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right], \quad G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \quad (10)$$

We define $\pi_{\theta}(a_t | s_t)$ as the probability of taking action a at time t , given the current state s at time t with parameter θ . The right-hand side of the policy gradient equation is a sum over states weighted by how often the states occur under the target policy π . Here, G_t denotes the return, incorporating the discount factor γ .

Policy gradient methods are utilized to optimize the parameter θ , to maximize the scalar performance measure $J(\theta)$. The parameter θ is adjusted primarily towards directions that promote actions yielding the highest return G , while

discouraging actions that are frequently selected but do not provide optimal returns.

The update formula for the parameters is given by:

$$\theta_{t+1} = \theta_t + \alpha_t G_t \frac{\nabla_{\theta} \pi_{\theta}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} \quad (11)$$

It is important to note that the gradient of the log-probability, $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$, is equivalent to the ratio $\frac{\nabla_{\theta} \pi_{\theta}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)}$. This relationship facilitates a more efficient computation of the gradient required for parameter updates, thereby enhancing the learning process under the policy gradient methods.

In Mask-PPO, the method of masking the logits for invalid actions involves replacing the logits of these actions with a large negative number M to avoid sampling invalid actions. To achieve this, a masking function, denoted as $\text{mask}: \mathbb{R} \rightarrow \mathbb{R}$ is used to represent the masking process, and the re-normalized probability distribution is calculated accordingly. The policy network outputs logits $l(s)$ corresponding to different actions. The masking function $\text{mask}(l(s))$ modifies the logits as follows:

$$\text{mask}(l(s))_i = \begin{cases} l_i & \text{if } a_i \text{ is valid in } s \\ M & \text{otherwise} \end{cases} \quad (12)$$

The policy resulting from applying the mask, denoted as $\pi_{\theta'}(\cdot | s)$, is as follows:

$$\pi'_{\theta}(\cdot | s) = \text{soft max}(\text{mask}(l(s))) \quad (13)$$

The "mask" function is evidently either an identity function or a constant function for the elements in the logits which are differentiable. So the policy $\pi_{\theta'}(\cdot | s)$ remains differentiable with respect to the policy parameters θ . Thus, Eq. 10 is updated according to the invalid action policy gradient shown in Eq. (14).

$$g_{\text{invalid action policy}} = \mathbb{E}_{\tau} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi'_{\theta}(a_t | s_t) G_t \right], \quad G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \quad (14)$$

The resulting gradient update follows the same principles as in standard policy gradient methods. With the expansion of the invalid action space, Mask-PPO still maintains good scalability and effectiveness.

The research goals in introducing Mask-PPO are to explore novel avenues for improving training dynamics, promoting efficient policy updates, and achieving enhanced model adaptability. This work contributes to the ongoing development of reinforcement learning methodologies.

Preference learning

Preference learning has emerged as a powerful framework in machine learning, where the goal is to induce predictive models based on preferences rather than absolute labels or rewards. Unlike conventional supervised learning tasks that rely on scalar target values, such as those in classification or regression, preference learning uses pairwise comparisons to express the relative desirability of different options. These comparisons are typically expressed as preferences between two trajectories. This type of learning is particularly useful in reinforcement learning (RL) settings where the specification of a reward function is challenging, ambiguous, or prone to exploitation.

In preference learning, the goal is to find a policy that maximizes the likelihood of producing trajectories that are consistent with preferences. These preferences are represented as binary comparisons: $\tau_i \succ \tau_j$, indicating that trajectory τ_i is preferred over trajectory τ_j .

The key objective of trajectory preference learning is to learn a policy π that generates trajectories that align with the given preferences. It offers a more flexible form of feedback than traditional scalar rewards. In many cases, it may be difficult or impractical to define an explicit reward function that accurately captures the desired behavior of an agent. Preference learning, however, allows for the comparison of trajectories based on qualitative factors that may be difficult to quantify numerically.

The learning process typically involves constructing a preference model. This model can be trained to predict the probability that one trajectory is preferred over another. In mathematical terms, the preference model $P(\tau_i \succ \tau_j)$ predicts the likelihood that trajectory τ_i is preferred over trajectory τ_j . This model is then used to guide the agent's policy updates.

The most common approach to preference learning in reinforcement learning is to represent the preference model as a reward predictor. Given two trajectories τ_i and τ_j , the reward predictor assigns a score to each trajectory based on an estimated reward function. The trajectory with the higher score is deemed preferable, following the Bradley–Terry model [42]:

$$P(\tau_i \succ \tau_j) = \frac{e^{r(\tau_i)}}{e^{r(\tau_i)} + e^{r(\tau_j)}} \quad (15)$$

where $r(\tau)$ is the predicted reward of trajectory τ . The model is trained by minimizing a loss function that captures the discrepancy between the predicted preferences and the target preferences.

The primary advantage of preference learning is that it enables agents to learn without explicit reward signals. This feature is particularly beneficial in many practical applications, especially in complex workshop scheduling tasks

where defining rewards can be challenging. The application of preference learning provides a flexible and adaptive approach capable of handling complex environments. It contributes to increased production efficiency, reduced resource waste, and improved scheduling outcomes in dynamically changing production environments.

Proposed methods for FJSP

This section introduces the transformation of the FJSP problems into a RL problem and establishes the FJSP scheduling framework. Subsequently, the core components of the framework and key algorithms are elaborated in detail.

Problem transformation

Jobshop information

Before executing the workshop scheduling algorithm, it is necessary to define the specific information of the job workshop to determine key parameters such as processing time and requirements for each job. This step is crucial for the rational allocation and scheduling of tasks. To achieve this goal, we can represent the processing information of the job workshop in matrix form, as shown below:

$$\begin{bmatrix} N_1 \\ \vdots \\ N_p \\ \vdots \\ N_n \end{bmatrix} = \begin{bmatrix} M_{11}, time_{11}; \dots; M_{1q}, time_{1q}; \dots; M_{1m}, time_{1m} \\ \vdots \\ M_{p1}, time_{p1}; \dots; M_{pq}, time_{pq}; \dots; M_{pm}, time_{pm} \\ \vdots \\ M_{n1}, time_{n1}; \dots; M_{nq}, time_{nq}; \dots; M_{nm}, time_{nm} \end{bmatrix} \quad (p \leq n; q \leq m) \quad (16)$$

where p represents the sequence number of the workpiece to be processed, q represents the sequence number of the machine to be processed on, $time_{pq}$ represents the processing time of the p -th workpiece on the q -th machine, n represents the total number of pending workpieces, m represents the total number of machines, n_p represents the p -th workpiece to be processed, and M_{pq} represents the processing of the p -th workpiece on the q -th machine.

State feature expression

The state feature is directly fed into PPO for training to obtain corresponding scheduling actions, so the quality of the state feature directly affects the quality of scheduling results. If the state feature is incomplete, even the best reinforcement learning algorithm cannot achieve good scheduling results. Similarly, if there is too much or redundant state feature information, it can slow down the training process.

Before defining the state feature for the workshop job environment, first, we traverse the workshop job feature to obtain the following values: *max_time_jobs* represents maximum cumulative scheduling time among all jobs, *max_time_op* represents maximum scheduling time for all operations, *sum_op* represents cumulative scheduling time for all operations of all jobs, and *machines* represents number of machines required for the current job.

These values are static data and remain constant throughout the scheduling process. A set of 7-dimensional vectors is defined to represent the state feature at each moment. These state information vectors are dynamic data that continuously change as the scheduling process advances. Each time the state feature is updated, a different elapsed timestamp represented by *difference* is obtained, allowing the calculation of the time interval since the last state update.

(1) State[0]

The state indicating whether the current job can be executed, which can expedite the training process of the Mask-PPO algorithm. Using a mask can directly shield jobs that are definitely unable to execute, preventing the training process from being aimless.

$$state[0] = \begin{cases} \text{false} & \text{indicates that the job can't be executed} \\ \text{true} & \text{indicates that the job can be executed} \end{cases} \quad (17)$$

(2) State[1]

The normalized remaining time of the current operation, meaning the normalization of the remaining time of the current operation to be within the range [0–1], aiming to enhance subsequent optimization learning.

$$state[1] = \frac{\max(0, time_left_current_op - difference)}{max_time_op} \quad (18)$$

where *time_left_current_op* represents the remaining time of the current operation; *difference* represents the time interval from the last state update to the current timestamp.

(3) State[2]

The percentage of completion of the current operation

$$state[2] = \frac{time_step_job}{machines} \quad (19)$$

where *time_step_job* represents the current index of the operation being executed.

(4) State[3]

The normalized total remaining time of the current job, meaning normalizing the total remaining time of the current job to be within the range [0, 1], to enhance subsequent optimization learning performance.

$$state[3] = \frac{total_perform_op_time_jobs}{max_time_jobs} \quad (20)$$

$$\begin{aligned} total_perform_op_time_jobs \\ = total_perform_op_time_jobs_o \\ + \min(difference, time_left_current_op) \end{aligned} \quad (21)$$

where *total_perform_op_time_jobs* represents the total remaining time of the current job, and *total_perform_op_time_jobs_o* represents the total remaining time of the previous processed job.

(5) State[4]

The normalized available time for the next operation's required machine, meaning normalizing the available time for the machine required by the next operation to be within the range [0–1], to enhance subsequent optimization learning performance.

$$state[4] = \frac{\max(0, time_until_available_machine - difference)}{max_time_op} \quad (22)$$

where *time_until_available_machine* represents the available time for the required machine. If *time_until_available_machine—difference* ≤ 0, it indicates that the previously occupied machine is now idle and can be used by the next operation of the current job.

(6) State[5]

The normalized idle time after the completion of the previous operation.

$$state[5] = \frac{difference - time_left_current_op}{sum_op} \quad (23)$$

(7) State[6]

The normalized cumulative idle time; this state variable is essentially the cumulative sum of *state*[5], which is not reset during the process.

$$state[6] = old.state[6] + state[5] \quad (24)$$

where *old.state*[6] represents the normalized cumulative idle time after the completion of the previous operation.

Considering both speed and accuracy, we select the aforementioned seven workshop job environmental state information. Due to the small dimensionality of the state vector, it ensures the computational speed of the neural network. Additionally, these seven state feature elements are representative and facilitate the easy construction of other commonly used scheduling rules.

Action definition

The action $a_t \in A_t$ is an operation to select the preferred job to dispatch according to the policy at each timestep t . In the FJSP environment, an action represents a job allocation. The original action space $J = \{J_0, J_1, \dots, J_{n-1}, nope\}$, where n represents the number of jobs, includes a "nope" action which represents no allocation. This "nope" action is primarily due to the following reasons:

- (1) Sometimes there are no available machines to execute the corresponding job operation.
- (2) Delaying job allocation at the current moment might be a preferable policy compared to immediate execution.

Since the action space in Mask-PPO is pruned each time the environment returns an observation, this pruning masks out invalid actions, including:

- (1) Jobs currently in progress.
- (2) Jobs that have already been completed, but whose next operation requires a machine that is currently in use.

Reward function

In DRL algorithms, the reward function is used to assess the performance of an agent after taking specific actions in the environment. It guides the agent to adjust its strategy during the learning process to achieve better performance.

Although the rewards returned by the FJSP may not accurately reflect the quality of the scheduling policy, the relative magnitudes of the cumulative rewards between the fragment pairs remains highly accurate. Therefore, in the PMBP algorithm, the rewards returned by the environment are primarily

used for preference learning, training the optimal preference learning model. The detail reward calculation process is as followed.

Step 1. Definition of the reward function. With makespan as the scheduling objective, the reward function for this algorithm is established as follows:

$$R(t) = T_q - idle(t_q, t_{q+1}) \quad (25)$$

where, *idle*($\cdot$) represents the idle time between t_q and t_{q+1} ; T_q denotes the completion time required for the current operation; t_q indicates the moment when the q -th operation of the p -th ($p \leq n$) job starts; t_{q+1} represents the moment when the $q+1$ -th operation of the p -th ($p \leq n$) job starts; if the idle time between two adjacent operations is larger, r_t is smaller, and the accumulated reward value is also smaller, indicating negative feedback. This implies that a larger idle time corresponds to a lower machine utilization rate, resulting in a larger makespan.

Step 2. Calculation of the cumulative reward value. By continuously sliding to solve the cumulative reward value with an additional discount factor, the formula is as follows:

$$R_t = \gamma R_{t-1} + r_t \quad (26)$$

where, γ represents the discount factor; R_{t-1} represents the cumulative reward value at time $t-1$; R_t represents the cumulative reward value at time t ; r_t represents the reward value of the initial reward function output at time t .

Step 3. Calculation of the mean and variance of reward values. The algorithm employs a recursive formula for solution, and the formula is as follows:

$$A_t = A_{t-1} - \frac{R_t - A_{t-1}}{t} \quad (27)$$

$$V_t = \frac{t-1}{t^2} (R_t - A_{t-1})^2 + \frac{t-1}{t} V_{t-1} \quad (28)$$

where, A_t represents the mean of the reward values of the initial reward function at time t , and V_t represents the variance of the reward values of the initial reward function at time t .

The instantaneous reward value r_t output by the workshop operation environment at time t is divided by the standard deviation and input into the agent. The formula is as follows:

$$r_{tb} = \frac{r_t}{\sqrt{V_t}} \quad (29)$$

where, r_{tb} represents the reward value of the initial reward function after standard deviation processing; *sqr*($\cdot$) denotes the square root operation.

Step 4. Clipping. The standard deviation-processed reward values of the initial reward function are clipped using

a clipping function, restricting r_t to the range $[-1, 1]$. The formula is as follows:

$$r_{ty} = \text{clip}(r_{tb}, -1, 1) \quad (30)$$

Overall framework

In this paper, a novel framework is proposed as Fig. 1. The framework comprises several key components, each playing an important role in the learning process.

When the agent interacts with the FJSP environment, a collection of trajectories is generated, with each trajectory representing a sequence of states, actions, and rewards. The trajectory buffer is responsible for storing these trajectories, facilitating experience replay, a technique that allows the agent to learn from past experiences and improve learning efficiency.

Subsequently, a random fragment querying mechanism is introduced to decompose trajectories into smaller, more manageable fragments. These queries are used to elicit feedback from the environment. The Gather preferences step is dedicated to collect qualitative feedback or preferences which is crucial for aligning the agent's policy with the nuanced goals of FJSP. The fragment queries and the accompanying preferences are stored into fragment dataset.

Afterwards, preferences and a pair of fragments are collected to train a reward net (64×64) that can predict or approximate the underlying preference function. The cross-entropy loss function is used to minimize the difference between the predicated preferences and the actual preferences.

Next, two independent sets of algorithms for sampling and training are integrated. PPO and random policy are utilized to strike a balance between exploration and exploitation by clipping the policy update, preventing excessive deviation from the current policy. Similarly with Mask-PPO, it masks out actions that are not feasible in the current state, effectively pruning the action space and ensuring that the agent's policy is constrained to valid scheduling decisions. The utilized reward values are not those directly returned by the FJSP environment, but rather the predicted values outputted by the above reward model trained through preference learning. This approach enhances the agent's learning guidance.

Trajectory preference elicitation

In order to generate diverse trajectories, sampled preferences must differ from existing trajectories. Besides, trajectories should also contain useful information about the transition function for optimal policy computation.

Initially, a PPO based policy is proposed for trajectory sampling, and Mask-PPO is chosen for final training. The reason for employing different strategies during sampling and training is that Mask-PPO tends to mask numerous actions, resulting in a lack of diversity in observation states, which, in turn, compromises the generalization capability of the reward model. On the contrary, PPO facilitates better exploration of the environment, generating more diverse trajectories, thereby contributing to the reward model training. We over-sample 10% transitions to avoid having the same transition in too many fragments since fragments are sampled with replacement.

In addition, two sampling policies are employed: PPO and Random Policy. 10% of trajectories are generated partially randomly rather than only by the PPO based policy sampling. The sampling process involving exploration consists of the following steps:

Step 1: Perform PPO policy sampling without exploration.

Step 2: When 90% of the total sampling is reached, the exploration process starts. There is a certain probability (*switch_prob*) of transitioning from the current policy. Otherwise, it remains unchanged. In our algorithm, *switch_prob* is set to 50%.

Step 3: If a transition occurs, there is a certain probability (*random_prob*) of selecting between the PPO and Random Policy. In the algorithm, *random_prob* is set to 50%.

The above exploration strategy further enhances the exploration. The sampling algorithm process is shown in Fig. 2.

Each fragment is part of a single trajectory and has a fixed length. The trajectories shorter than the desired fragment length are discarded. The complete procedure of sampling fragments of trajectories is summarized in Algorithm 1.

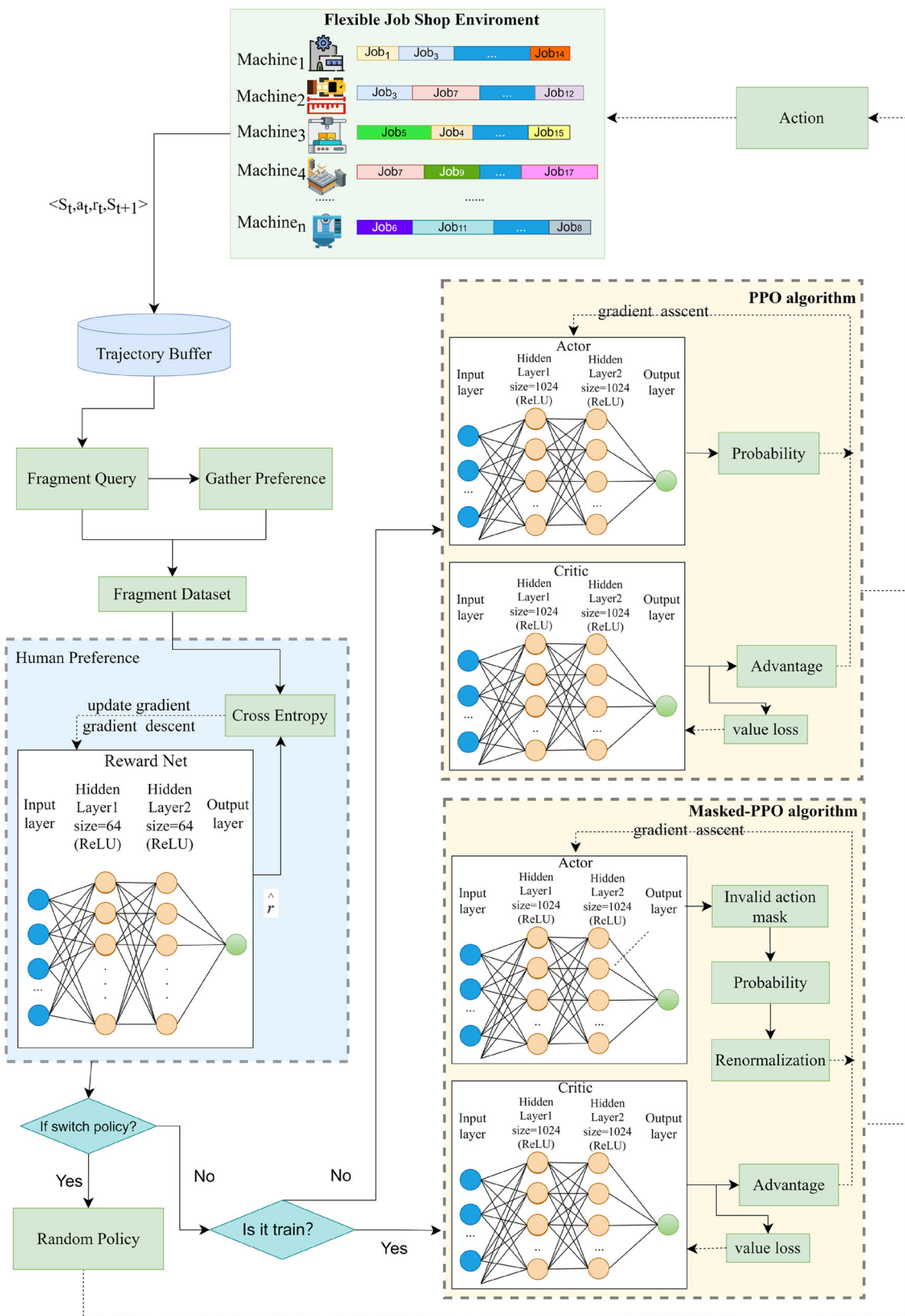


Fig. 1 Overall framework

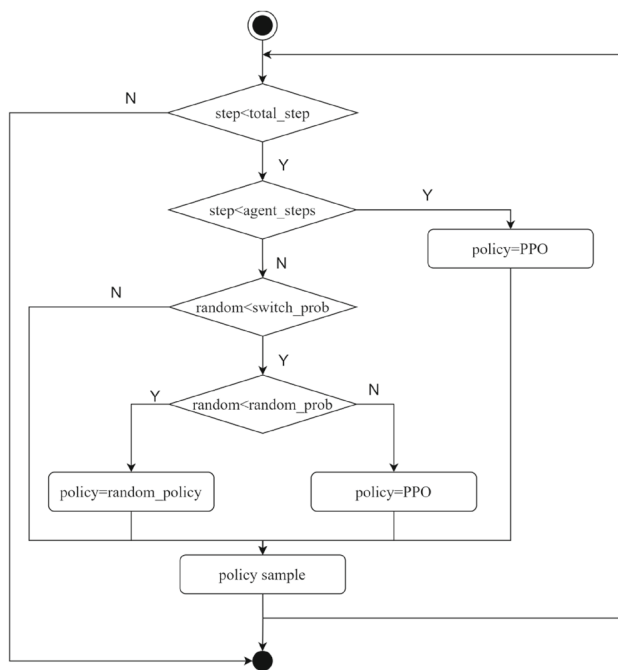


Fig. 2 Sampling policy switching mechanism

The synthetic preferences are computed using ground-truth environment rewards which is the operation time percentage. In calculating the sum of reward values $\sum r(\sigma)$ for fragments, we consider future rewards to be equally important as current rewards. Thus, we set *discount_factor* to 1 because the impact of both current and future actions on the total scheduling time is considered to be the same. Pairs of fragment rewards are compared using the sign function to calculate the probability that fragment 1 is preferred over fragment 2.

$$\mu = \frac{\text{sign}(\sum(r(\sigma^0)) - \sum(r(\sigma^1)) + 1)}{2} \quad (31)$$

Typically, there are a total of 3 values:

- (1) If the result is equal to 1, it indicates that fragment 1 is preferred over fragment 2.
- (2) If the result is equal to 0, it indicates that fragment 2 is preferred over fragment 1.
- (3) If the result is equal to 0.5, it indicates that the preference for fragment 1 is equal to fragment 2.

Algorithm 1: Fragment sampling

Input: number of timesteps *fragment_length* per fragment; number of preferences *trajectories* in each iteration; number of comparisons *num_pairs* in each iteration

```

1: Initialize valid_trajectories with empty list
2: for each item in trajectories do // filter out trajectories that are too short
3:   if length(item) < fragment_length:
4:     continue
5:   end if
6:   append item to valid_trajectories
7: end for
8: Initialize fragments with empty list
9: calculate weights for the valid_trajectories based on its length
10: for each pair_index in num_pairs do
11:   select a trajectory randomly from valid_trajectories based on weights / sum(weights)
12:   // generate a random start_index within the trajectory:
13:   start_index ← random_integer(0, length(trajectory) - fragment_length)
14:   end_index ← start_index + fragment_length
15:   if end_index > length(trajectory):
16:     end_index ← length(trajectory)
17:   end if
18:   create a fragment from a slice of trajectory range [start_index, end_index)
19:   append fragment to fragments
20: end for
21: Initialize paired_fragments with empty list
22: for i in length(fragments) step 2 do
23:   append (fragments[i], fragments[i+1]) to paired_fragments
24: end for
25: return paired_fragments
  
```

Then the obtained fragments (σ^0, σ^1) and computed preferences probabilities compose μ the dataset $\mathcal{D} = (\sigma^0, \sigma^1, \mu)$ used to train the reward model.

Learning a preference model

Preference model

The reward network is constructed using a multi-layer perceptron (MLP), which takes the state and action as inputs and outputs a scalar reward value. The network has two hidden layers, each of size 64×64 . The preference model takes the predicted reward output as input to compute the probability that the first fragment is preferred over the second. The probability is modeled as:

$$\hat{P}[\sigma^0 \succ \sigma^1] = \frac{1}{1 + \exp(\text{Clip}(\sum \hat{r}(o_t^1, a_t^1) - \sum \hat{r}(o_t^0, a_t^0), -\varepsilon, +\varepsilon))} \quad (32)$$

Rather than directly applying Eq. (15), we introduce a novel approach by normalizing both the numerator and denominator with $\sum \hat{r}(o_t^0, a_t^0)$ and integrates a *Clip function* to constrain the difference within the range $[-\varepsilon, +\varepsilon]$, thereby preventing overflow, particularly during backpropagation.

This technique improves the robustness of the learning process. By applying the normalization, we estimate the probability that $\hat{r}(o_t^0, a_t^0) > \hat{r}(o_t^1, a_t^1)$ turns out true, where $\hat{r}(o_t^1, a_t^1)$ represents the reward predicted by the reward function for the state o_t and action a_t in the fragment σ^1 . The exponential function $\exp(\cdot)$ ensures that the predicted rewards are transformed into non-negative values, allowing them to be interpreted as probabilities.

Loss function

The preferences are represented in the dataset as triples $(\sigma^0, \sigma^1, \mu)$, where (σ^0, σ^1) are trajectory fragments and μ is the target probability label indicating which fragment is preferred. Specifically, $\mu \in \{0, 1, 0.5\}$. The predicted probability $\hat{P}[\sigma^0 \succ \sigma^1]$ from the reward model and target label μ are used to compute the binary cross-entropy loss, which quantifies the difference between the predicted preference probability and the actual observed preference. The loss function L can be expressed as follows:

$$L = - \sum_{(\sigma^0, \sigma^1, \mu)} \mu(0) \log \hat{P}[\sigma^0 \succ \sigma^1] + \mu(1) \log \hat{P}[\sigma^0 \prec \sigma^1] \quad (33)$$

This loss function encourages the model to maximize the predicted probability for the correct preference label. When the model's prediction aligns with the true preference, the loss is minimized. Conversely, when the prediction deviates from the true preference, the loss increases.

The gradient of the binary cross-entropy loss with respect to the predicted rewards $\hat{r}(\sigma^0)$ and $\hat{r}(\sigma^1)$ can be derived as follows:

For $\hat{r}(\sigma^0)$:

$$\frac{\partial L}{\partial \hat{r}(\sigma^0)} = - \left(\frac{\mu(0)}{P(\sigma^0 \succ \sigma^1)} - \frac{\mu(1)}{P(\sigma^0 \prec \sigma^1)} \right) \cdot \frac{\partial P(\sigma^0 \succ \sigma^1)}{\partial \hat{r}(\sigma^0)} \quad (34)$$

Similarly, for $\hat{r}(\sigma^1)$:

$$\frac{\partial L}{\partial \hat{r}(\sigma^1)} = - \left(\frac{\mu(0)}{P(\sigma^0 \succ \sigma^1)} - \frac{\mu(1)}{P(\sigma^0 \prec \sigma^1)} \right) \cdot \frac{\partial P(\sigma^0 \succ \sigma^1)}{\partial \hat{r}(\sigma^1)} \quad (35)$$

These gradients are then used to update the parameters of the reward network during backpropagation. The exponential nature of the logistic function ensures that the reward predictions are adjusted in a way that increases the likelihood of correctly predicting the preferences.

In each training iteration, the agent is updated using the PPO algorithm. The reward model, trained in prior steps, serves as a preference-guided signal for updating the policy. The goal is to align the agent's behavior with the learned preferences. The iterative process refines the agent's policy by incorporating feedback from the reward model.

Advantages of preferences learning

Once trained, the reward model can be used as a reward predictor during subsequent agent training phases. This allows the reward predictor to replace the original environment-provided rewards, offering several advantages:

1. **Handling partial information:** In the FJSP, obtaining precise and accurate reward values for all possible state-action pairs can be challenging. Preference learning allows the model to learn from relative preferences, even

when absolute reward values are unknown or difficult to specify.

2. **Robustness to reward scaling:** Since the model learns based on relative preferences between different experiences, it is less sensitive to changes in the absolute magnitude of the reward values. The learned preferences are scale-invariant, ensuring robustness against variations in the reward function.
3. **Generalization to new FJSP tasks:** The model does not rely on specific reward values that are environment-dependent. Instead, it learns general preferences, which can be applied across different FJSP contexts. This enhances the model's ability to generalize to new and unseen scheduling tasks.

Agent training

After the reward model is trained, the agent is further trained using Mask-PPO. Instead of applying an invalid action penalty, which discourages the selection of invalid actions by returning a negative reward. We employed a technique called invalid action masking to prevent the execution of invalid actions in a given job state.

When invalid action masking is applied, if “masking out” is true for an invalid action, the corresponding logit value is preserved. Otherwise, the logit value is set to a large negative constant (e.g., $M = -1e-8$), resulting in a probability near zero. If no masks are applied, any previously applied masking is removed, and the original logits are restored. Consequently, after the softmax function, the probabilities of invalid actions become close to zero, preventing them from being selected.

Following this masking process, the policy gradient is calculated using the masked action probability distribution. This ensures that only the valid actions contribute to the gradient update, effectively improving the policy by excluding invalid actions.

The gradient update rule remains consistent with the standard policy gradient approach, but it is now applied only to the policy that produces valid actions. Then, the action probability distribution undergoes renormalization to ensure that the sum of probabilities for all valid actions equals 1. After an action is executed and the corresponding reward is observed, the policy gradient is calculated based on the masked action probability distribution. This means that only the gradients corresponding to valid actions are used to update the policy network's parameters.

This update method ensures the effectiveness of the policy gradient calculation by focusing exclusively on the valid actions that the agent performs. The gradient generated through invalid action masking corresponds to a valid policy gradient. It works because the masking process can be considered a state-dependent differentiable function applied during the calculation of the action probability distribution. It proves to scale well as the number of invalid actions increase.

In contrast to traditional way of RL training, we leverage the reward model trained in the previous step as a reward predictor. This eliminates the dependency on environment-based rewards, enabling a more robust and adaptable training process. The complete training process of PBMP is summarized in Algorithm 2.

Algorithm 2: PBMP algorithm

Input: number of environment interaction steps $total_comparisons$; number of times to train the agent and then train the reward model against newly gathered preferences $num_iterations$

1: Initialize RewardNet with MLP

2: Wrap environment $venv$ with RewardNet

3: Compute $schedule_list$ of $total_comparisons$ with normalized hyperbolic

$$\frac{1}{t} (t = 0, \frac{1}{iterations}, \frac{2}{iterations}, \dots, 1)$$

4: for $pairs$ in $schedule_list$ do

$iteration \leftarrow$ position of $schedule_list$

5: Sample trajectories $\tau_t = ((o_t^0, a_t^1), (o_t^0, a_t^1), \dots, (o_t^N, a_t^N))$ with PPO

6: Extend τ_t with explorations (random switch between PPO and a random policy)

7: Select random fragment pairs (σ^0, σ^1) of τ_t

8: Compute probability:

$$\mu \leftarrow -\frac{sign(r(\sigma^0) - r(\sigma^1)) + 1}{2}$$

9: Gather Synthetic Preferences $d = (\sigma^0, \sigma^1, \mu)$

10: Add Preferences d to Dataset D

11: if $iteration == 0$

epochs $\leftarrow$ epochs $\times$ epoch_multiplier

end if

14: for each $epoch$ in $epochs$ do

for each fragment pair in D do

Computes the Boltzmann rational probability

$$17: \quad P[\sigma^0 \succ \sigma^1] = \frac{1}{1 + \exp(\text{Clip}(\sum \hat{r}(o_t^1, a_t^1) - \sum \hat{r}(o_t^0, a_t^0), -\varepsilon, +\varepsilon))}$$

18: Measure Binary Cross Entropy Loss:

$$19: \quad L = -\sum_{(\sigma^0, \sigma^1, \mu)} \mu(0) \log \hat{P}[\sigma^0 \succ \sigma^1] + \mu(1) \log \hat{P}[\sigma^0 \prec \sigma^1]$$

20: Renormalize the loss to the averaged over the whole batch size

$$21: \quad L^* = \frac{\text{length}(\text{fragment_pairs})}{\text{batch_size}}$$

22: Minimize the L via gradient descent algorithm

23: end for

24: end for

25: for each timestep t do

Collect $(s_t, a_t, s_{t+1}, r(s_t))$ with action mask into rollout buffer R

27: for each epoch do

for batch size in R do

Compute advantage estimates A_t based on the current value function V

Update the policy by maximizing the PPO-Clip objective

Compute Loss

$$32: \quad Loss_{mask_ppo} = Loss_{clipped_surrogate} + ent_coef \times Loss_{entropy} + vf_coef \times Loss_{value}$$

Minimize the loss via gradient descent algorithm

end for

end for

36: end for

37: end for

Experiment and results

Experimental environment and parameter settings

The algorithm employed in this study is executed on the macOS 14.3.1 operating system, utilizing Python version 3.9.16 and PyTorch 1.13.1 environment. The computer hardware configuration includes an Intel Core i7-9750 CPU @2.60 GHz and 16 GB of RAM.

Optuna [43] is used to optimize the hyperparameters. It is an automatic hyperparameter optimization software framework, specifically designed for machine learning. The PMBP algorithm in this study primarily consists of two modules: preference learning and Mask-PPO. Therefore, the hyperparameters are mainly divided into two parts, as shown in Table 2. It has been demonstrated that the following hyperparameters result in superior scheduling policies and improved generalization capabilities in the experiments.

Experimental results and analysis

In this section, we present a series of benchmark experiments aimed at evaluating performance and comparing various algorithms. To assess the effectiveness of the PBMP algorithm, we have formulated the following scenarios for simulation analysis. The experiments are designed to explore the following:

- (1) The correlation between the output reward from the preference learning model and the resulting makespan is investigated.
- (2) PBMP is compared to the prior methods regarding makespan.
- (3) The contribution of each key component in PBMP is analyzed.

Correlation analysis

The primary objective of this experiment is to analyze the correlation between the output reward values from the preference learning model and the resulting makespan in the generated schedules. The Pearson correlation coefficient (PCC) is utilized to quantify the strength and direction of the correlation.

The PCC, denoted as $r_{x,y}$ is commonly employed to assess the linear correlation between two sets of data. It is defined by the formula:

$$r_{x,y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \quad (36)$$

where x_i and y_i represent individual data points. This coefficient ranges from -1 to 1 , with positive values indicating positive correlation, negative values denoting negative correlation, and 0 indicating no linear correlation between the datasets.

We employed seven sets of instances of varying sizes. For each instance, we computed the PCC between the total rewards and the makespans of multiple trajectories. The comparison of PCC values between the preference learning model and the environment is shown in Fig. 3. As the PCC value approaches -1 , the color on the colorbar's left side indicates a stronger negative correlation between the output of rewards and makespan. This implies that the reward value better aligns with the actual makespan, resulting in enhanced performance.

The experiment reveals that the Pearson correlation of the preference learning model demonstrates a higher negative correlation compared to that of the environment, indicating that preference learning can effectively capture the complex relationship between the reward and makespan.

Although the negative correlation diminishes with the increase in instance size for both methods, the preference learning model demonstrates significantly lower attenuation, maintaining a strong negative correlation consistently.

Hence, preference learning can serve as a viable reward mechanism for reinforcement learning training in FJSP, leading to more efficient and effective scheduling solutions.

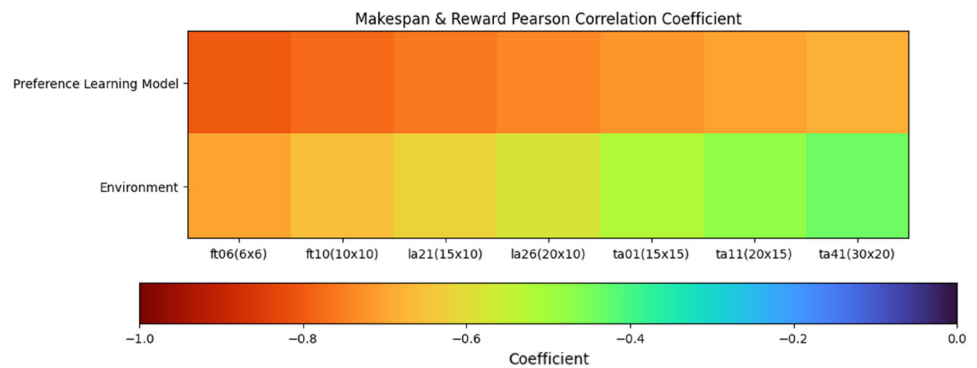
Benchmarking prior methods

We conducted comparisons between our proposed algorithm and traditional scheduling rules, OR-Tools, and DRL algorithms to demonstrate its effectiveness from various perspectives. The following is a list of the comparative algorithms adopted in our experiment:

1. Four dispatching rules: These heuristic methods are commonly used in workshop scheduling to formulate decisions for effective job scheduling. Based on simple heuristic principles, these rules make scheduling decisions considering the current workshop status and job characteristics. We selected the following four strategies: longest processing time (LPT), shortest processing time (SPT), first in first out (FIFO), and last in last out (LIFO).
2. OR-Tools: OR-Tools is an open-source software suite developed by Google, providing a collection of tools and libraries for solving various operations research and optimization problems. It offers effective tools and algorithms for workshop scheduling, covering various aspects such as precedence constraints, time windows, and resource constraints, making it a powerful tool for addressing job shop scheduling problems.

Table 2 Hyperparameters

Preference learning		Mask-PPO	
Hyperparameter	Value	Hyperparameter	Value
n_timesteps	5e5	n_epochs	20
n_comparisons	5e3	n_timesteps	7e5
fragment_length	100	batch_size	512
preference_batch_size	2048	lr	lin_0.0001
num_iterations	10	clip_range	0.2
transition_oversampling	1.1	gamma	1.0
gather_switch_prob	0.5	gae_lambda	0.95
gather_random_prob	0.5	ent_coef	5e−4
reward_net_lr	1e-3	vf_coef	1.0
reward_net_batch_size	64	pi_net_hid_size	1024 × 1024
reward_net_epoch	3	vf_net_hid_size	1024 × 1024
reward_net_hid_size	64 × 64	shared_feature_extractor	False

Fig. 3 Makespan & reward Pearson correlation

3. DQN: Deep Q-Network (DQN) includes deep neural networks that approximate the Q-function, experience replay, and target networks. It is well-regarded as a fundamental algorithm in the field of deep reinforcement learning.

The instances were obtained from five different datasets, each comprising various sizes. Table 3 provides detailed information about the instances utilized in the experiment.

We initially conducted training on generated instances of small to medium sizes. All methods were evaluated over 500 episodes, and the experimental results along with comparisons with other algorithms are presented in Table 4. For each solution, $C_{\max}$ represents the maximum completion time among all jobs. Gap refers to the relative deviation of the makespan ($C_{\max}$) from the best possible makespan (which may not necessarily be optimal). The Gap method can use the following formula.

$$Gap = \frac{C_{\max} - UB}{UB} * 100\% \quad (37)$$

Table 3 Benchmarks

Benchmarks	Source
ft06(6 × 6), ft10(10 × 10), ft20(20 × 5)	Fisher and Thompson [44]
la01(10 × 5), la06(15 × 5), la21(15 × 10), la26(20 × 10)	Lawrence [42]
dmu11(30 × 15), dmu16(30 × 20)	Demirkol et al. [45]
swv11(50 × 10), swv16(50 × 10)	Storer et al. [46]
ta01(15 × 15), ta02(15 × 15), ta11(20 × 15), ta31(30 × 10), ta41(30 × 20), ta51(50 × 15)	Taillard [47]

Note: $n \times m$ represents n jobs and m machines

where UB represents upper bound.

1. The proposed PBMP algorithm is consistently delivers superior results across all instances compared to Dispatching Rules and RL. For Instance ft20, it surpasses SPT and DQN by 8.76 and 10.56% respectively in terms

Table 4 Experimental results with different algorithms

Instance	Scale		PBMP	Dispatching rules				OR-Tools	RL	Upper bound
				SPT	LPT	FIFO	LIFO			
ft06	6×6	C_{max}	55	88	77	68	70	55 (< 0.1)	55	55
		Gap	0.00%	60.00%	40.00%	23.64%	27.27%	0.00%	0.00%	–
ft10	10×10	C_{max}	930	1074	1295	1262	1281	930 (2.9)	980	930
		Gap	0.00%	15.48%	39.25%	35.70%	37.74%	0.00%	5.38%	–
ft20	20×5	C_{max}	1165	1267	1631	1488	1313	1165 (0.5)	1288	1165
		Gap	0.00%	8.76%	40.00%	27.73%	12.70%	0.00%	10.56%	–
la01	10×5	C_{max}	666	751	822	830	764	666 (< 0.1)	666	666
		Gap	0.00%	12.76%	23.42%	24.62%	14.71%	0.00%	0.00%	–
la06	15×5	C_{max}	926	1200	1125	1078	1031	926 (< 0.1)	926	926
		Gap	0.00%	29.59%	21.49%	16.41%	11.34%	0.00%	0.00%	–
la21	15×10	C_{max}	1222	1324	1451	1417	1479	1046 (64.3)	1262	1046
		Gap	16.83%	26.58%	38.72%	35.47%	41.40%	0.00%	20.65%	–
la26	20×10	C_{max}	1304	1498	1643	1479	1652	1218 (3.5)	1387	1218
		Gap	7.06%	22.99%	34.89%	21.43%	35.63%	0.00%	13.88%	–
ta01	15×15	C_{max}	1327	1462	1701	1830	1627	1231 (2.5)	1402	1231
		Gap	7.08%	18.77%	38.18%	48.66%	32.17%	0.00%	13.89%	–
ta02	15×15	C_{max}	1378	1446	1755	1526	1767	1244 (52.8)	1426	1244
		Gap	10.77%	16.24%	41.08%	22.67%	42.04%	0.00%	14.63%	–

Note: The data in the column “OR-Tools” enclosed in parentheses represents the runtime, measured in seconds. The runtimes for all other methods are within 10 s, hence not specified in the table

of C_{max} result. Similarly, for Instance la26, it outperforms FIFO and DQN by 13.42 and 6.37% respectively in terms of C_{max} result. These findings show that PBMP not only surpasses Dispatching Rules and RL, but also scales well, producing consistently better results as the instance size grows. This performance is particularly noticeable in instances like ft10 and la21, where PBMP’s C_{max} remains closer to the upper bound despite increased problem complexity, unlike other methods that exhibit larger performance gaps.

- When the instance size of the scheduling problem is small (6×6 , 10×5 , 15×5), the proposed PBMP algorithm can achieve optimal solutions similar to those found by OR-Tools compared to other methods. These results demonstrate that PBMP not only excels in smaller, well-constrained problem instances but also guarantees optimality, which is often missed by traditional methods.
- As the problem scale increases to medium-sized instances, PBMP’s scheduling results remain highly competitive, closely approximating OR-Tools. For Instance ta01 and ta02, it is only 7.08 and 10.77% higher than the optimal value. While dispatching rules like LIFO and RL methods experience a sharp increase in C_{max} , PBMP maintains consistently lower gap values, even as problem complexity increases. Moreover, PBMP

demonstrates significant efficiency in scheduling time, completing tasks much faster than OR-Tools, making it not only accurate but also computationally efficient for medium-scale problems.

- Each method within the dispatching rule exhibits significant variability in scheduling results, and no single method can consistently achieve good results within dispatching rules. In contrast, PBMP consistently provides some of the best results in terms of C_{max} across all instances. Even in large and complex instances like la26 and ta01, where other methods show significant performance degradation (e.g., FIFO with a 48.66% gap in ta01), PBMP remains one of the top performers, maintaining much lower gap values and demonstrating robust, reliable performance. The consistency of PBMP across small, medium instances highlights its adaptability and overall superiority in various scheduling environments.

Then, we conducted training on generated instances of large sizes. All methods were evaluated over 500 episodes, and the experimental results along with comparisons with other algorithms are presented in Table 5. Through extensive experimentation, the following conclusions can be reached.

Table 5 Experimental results in large-scale instances

Instance	Scale		PBMP	Dispatching rules				OR-Tools	RL	Upper bound
				SPT	LPT	FIFO	LIFO			
dmu11	30×15	$C_{\max}$	3995	4462	5282	5088	6797	3551 (600.0)	4138	3430
		Gap	16.47%	30.09%	53.99%	48.34%	98.16%	3.53%	20.64%	–
dmu16	30×20	$C_{\max}$	4608	4907	5841	5198	5902	3805 (600.0)	4614	3751
		Gap	22.85%	30.82%	55.72%	38.58%	57.34%	1.44%	23.01%	–
swv11	50×10	$C_{\max}$	3254	3714	4763	3784	3744	2983 (402.3)	3565	2983
		Gap	9.08%	24.51%	59.67%	26.85%	25.51%	0.00%	19.51%	–
swv16	50×10	$C_{\max}$	3048	3453	3416	3298	3252	2924 (3.8)	2924	2924
		Gap	4.24%	18.09%	16.83%	12.79%	11.22%	0.00%	0.00%	–
ta11	20×15	$C_{\max}$	1619	1865	2046	2018	1859	1372 (600.0)	1716	1357
		Gap	19.31%	37.44%	50.77%	48.71%	36.99%	1.11%	26.46%	–
ta31	30×15	$C_{\max}$	2278	2335	2417	2436	2417	1814 (600.0)	2286	1764
		Gap	29.14%	32.37%	37.02%	38.01%	37.02%	2.83%	29.59%	–
ta41	30×20	$C_{\max}$	2433	2499	2925	2981	2760	2087 (600.0)	2450	2005
		Gap	21.35%	24.64%	45.89%	48.68%	37.66%	4.09%	22.19%	–
ta51	50×15	$C_{\max}$	3187	3856	3880	3717	3391	2760 (399.2)	3263	2760
		Gap	15.47%	39.71%	40.58%	34.67%	22.86%	0.00	18.22%	–

Note: The data in the column “OR-Tools” enclosed in parentheses represents the runtime, measured in seconds. Due to the extended runtime of OR-Tools in such experiments, a maximum runtime limit of 600 s was set, and the computation was halted if it did not complete within that timeframe. The runtimes for all other methods are within 10 s and are therefore not specified in the table

1. The proposed PBMP algorithm demonstrates significant superiority over both dispatching rules and RL across all large-scale scheduling instances. For Instance swv11, PBMP outperformed SPT and DQN by 14.14 and 9.56%, respectively, in terms of $C_{\max}$ results. Similarly, for Instance ta51, PBMP outperformed FIFO and DQN by 16.63 and 2.31% respectively in terms of $C_{\max}$ results. These results clearly demonstrate PBMP’s robustness and ability to generalize effectively to large-scale problems.
2. OR-Tools often failed to find optimal solutions within a reasonable timeframe for more than half of the large-scale instances, with scheduling times significantly exceeding those of other scheduling methods. In contrast, PBMP consistently identified superior solutions with significantly shorter makespans, all while maintaining much lower computation times than OR-Tools. This efficiency makes PBMP particularly suited for large-scale, time-sensitive scheduling tasks where OR-Tools may struggle with extended computation times.
3. PBMP demonstrates remarkable stability in large-scale scheduling instances, with Gap values consistently fluctuating within a narrow range, typically around 25%. This stability is particularly evident in challenging instances such as swv11 and ta51, where the PBMP algorithm

maintained a relatively low Gap compared to dispatching rules, which showed significant variance in performance.

The experimental results illustrate that the PBMP algorithm outperforms both dispatching rules and RL across different scales of scheduling problems. Importantly, in large-scale scenarios, PBMP not only achieves better scheduling performance but also offers faster scheduling times when compared to OR-Tools, which often suffers from time constraints. These characteristics establish PBMP as a highly effective and scalable solution for addressing complex, large-scale scheduling challenges. Its capability to balance both solution quality and computational efficiency across diverse scheduling scenarios further underscores its potential as a promising approach that combines precision, efficiency, and adaptability.

In the subsequent experiments, Gantt charts illustrating the PBMP solutions for instances la21 and ta11 are presented in Figs. 4 and 5, respectively. Figure 4 shows the process of handling the 15×10 instance, while Fig. 5 shows the process of handling the 20×15 instance. In the Gantt chart, each color block corresponds to a specific job, with their respective start and end times indicated on the axis.

Upon examining the Gantt charts, it becomes apparent that the algorithm introduced in this paper optimizes the completion times for instances of varying scales, yielding values of

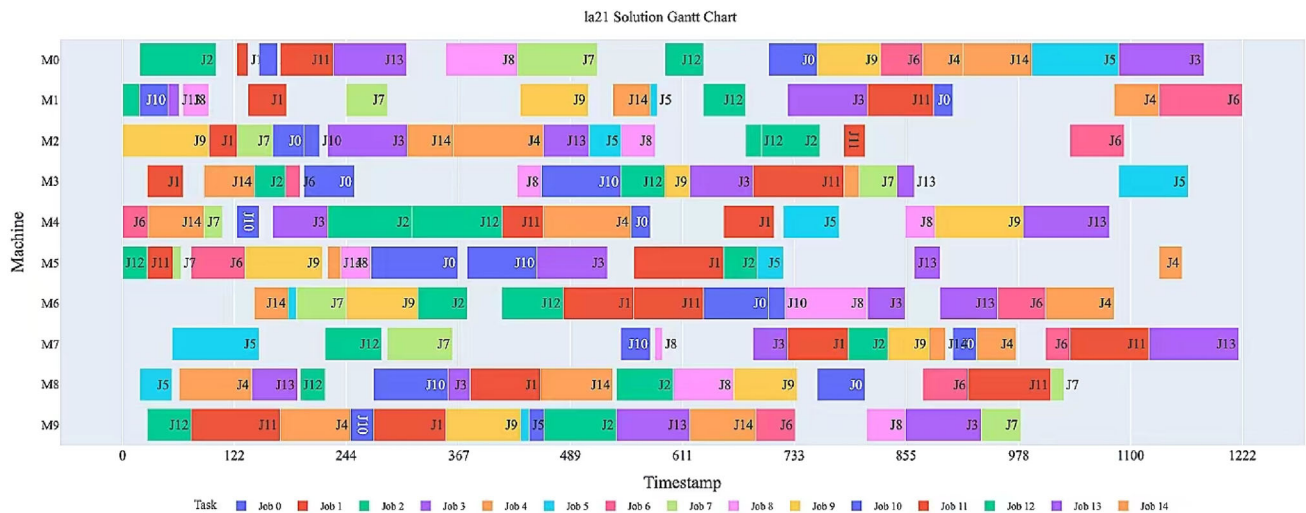


Fig. 4 Gantt chart of la21's optimal solution

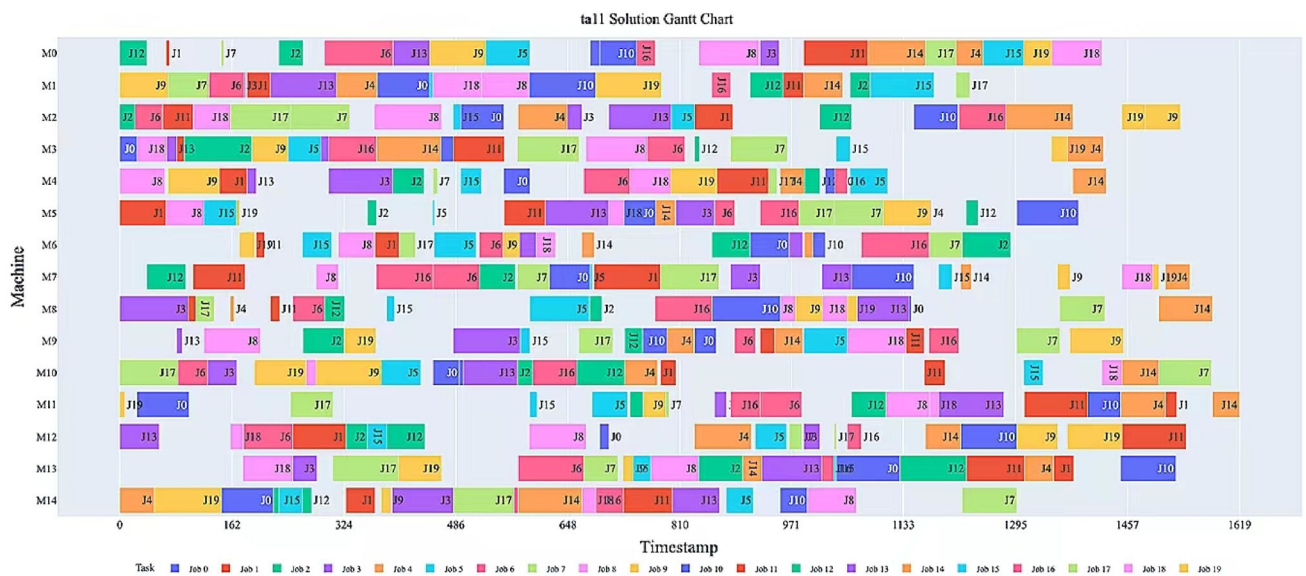


Fig. 5 Gantt chart of ta11's optimal solution

1222 and 1619. The algorithm facilitates more efficient job execution sequences, ensuring the timely completion of all tasks and maximizing the utilization of each machine. This provides persuasive evidence of the efficacy of the proposed scheduling approach.

Ablation experiment results

Ablation experiments are conducted to evaluate the contribution of each component within PBMP to its overall performance. By isolating and analyzing each key element, we can better understand its impact and importance, guiding future optimization efforts. These experiments provide insights into the relative importance of each component in

achieving superior performance. To assess the individual effects of each technique in PBMP, we examined the following modifications:

1. Softmax: Instead of the sign function, the softmax function was employed to compute synthetic preference probability.
2. No Exploration: No Exploration approach was employed throughout the sampling process, maintaining a consistent utilization of the PPO policy.
3. Mask-PPO: The training exclusively relied on Mask-PPO without incorporating preference learning.

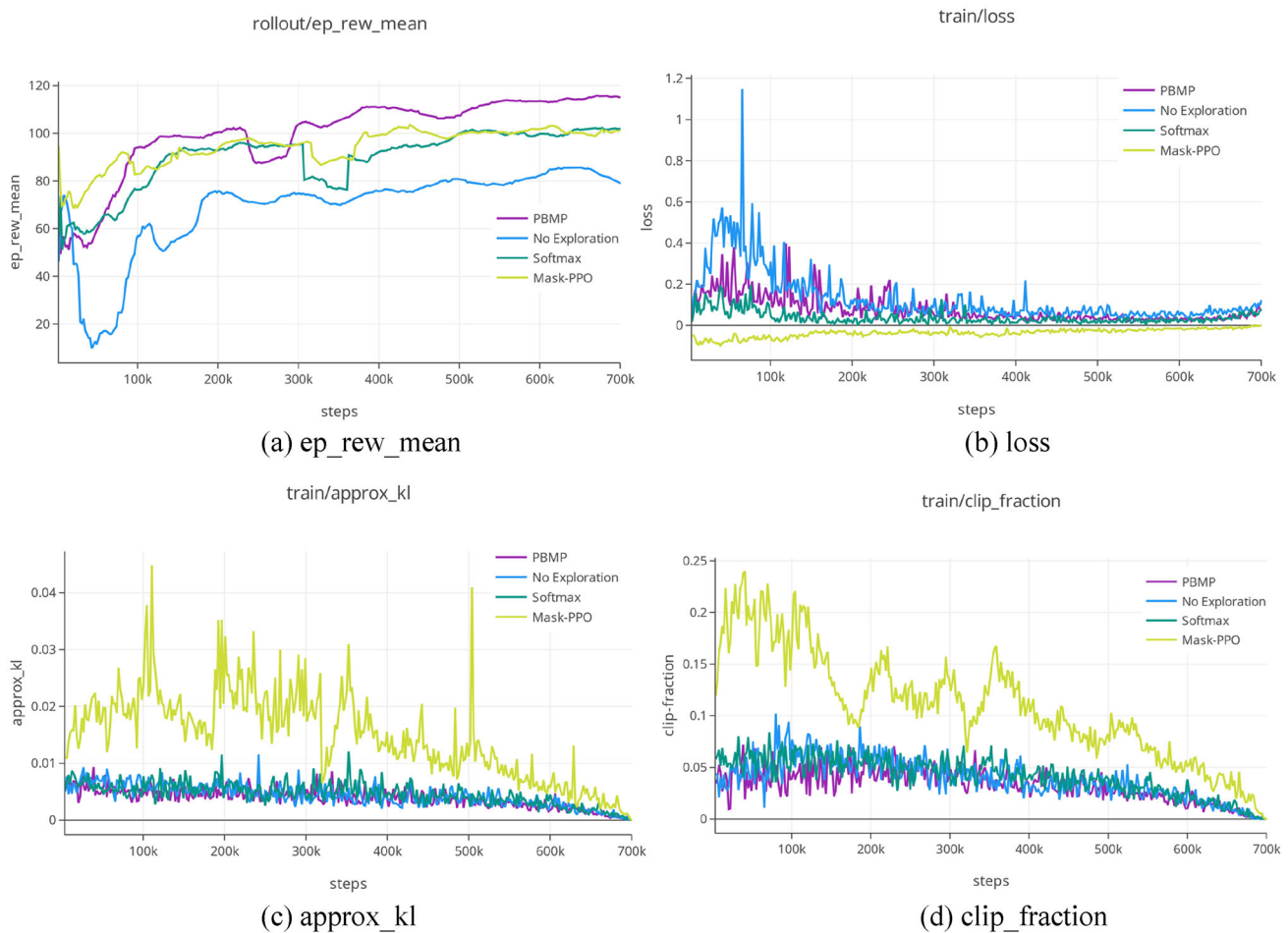


Fig. 6 Ablation experimental results

The key metrics analyzed in the ablation experiments are described as follows:

1. Mean reward per episode (ep_reward_mean): This metric indicates the average reward obtained per episode, reflecting the overall performance. Sustained high values of ep_reward_mean across various ablation scenarios underscore the algorithm's ability to consistently achieve high episode rewards.
2. Training loss (loss): This composite metric, comprising policy gradient loss and value loss, quantifies the difference between predicted outcomes and actual observations during training. A consistent reduction in training loss throughout the experiments signifies efficient learning and optimization within the algorithm.
3. Approximate KL divergence (approx_kl): This measures the difference between the current and previous policy distributions. Stability in this metric demonstrates controlled policy updates and smooth learning dynamics.
4. Clip fraction (clip_fraction): This measures how frequently policy updates are clipped to prevent overly

large changes that could destabilize learning. A lower clip fraction indicates more controlled and stable policy adjustments.

The ablation study clearly demonstrates the critical role of each key component within PBMP. The sign function in preference learning is essential for guiding the model towards optimal solutions, as evidenced by the weaker performance of the Softmax. PBMP's exploration strategy is crucial for discovering optimal scheduling policies, shown by the poor performance of the No Exploration. Additionally, preference learning significantly enhances policy stability and efficiency, with the Mask-PPO showing slower convergence and higher clipping without it. Together, these components enable PBMP to achieve faster convergence, lower training loss, and more stable updates compared to other methods.

In Fig. 6a, PBMP clearly outperforms the other methods by converging faster and achieving higher rewards. Replacing the sign function with the softmax function leads to reduced performance, underscoring the importance of the sign function in accurate preference learning. The No Exploration

significantly underperforms, highlighting the essential role of PBMP's exploration strategy in discovering optimal scheduling policies. The Mask-PPO, without preference learning, converges slower, demonstrating that preference learning is key to accelerating reward optimization.

In Fig. 6b, PBMP achieves lower and more stable training loss, while No Exploration exhibits higher variability, emphasizing the role of exploration in efficient learning. The Softmax struggles with higher loss, reinforcing that the sign function is crucial for guiding optimization. Mask-PPO lags in convergence, emphasizing the role of preference learning in reducing errors and improving optimization speed.

In Fig. 6c, PBMP maintains low and stable KL divergence, indicating smoother and more controlled policy updates. Both the Softmax and No Exploration show higher fluctuations, highlighting PBMP's superior exploration and preference learning mechanisms in ensuring stable updates. The Mask-PPO, without preference learning, exhibits less controlled updates, confirming the importance of preference-based stability.

In Fig. 6d, PBMP demonstrates a lower clip fraction, reflecting more controlled and precise policy updates. In contrast, both Softmax and No Exploration suffer from higher clipping rates, further underscoring PBMP's balance in exploration and exploitation. The Mask-PPO also experiences increased clipping without preference learning, demonstrating that preference learning contributes to more precise updates.

Conclusions and future work

In this research, the preference learning based Mask-PPO algorithm is proposed for the first time to solve the FJSP, representing a significant advancement in addressing complex scheduling tasks. The algorithm effectively overcomes the challenge of reward function design, a common bottleneck in applying DRL to job shop scheduling, and demonstrates superior performance in optimizing both scheduling efficiency and flexibility.

Firstly, the algorithm introduces a novel sampling policy switching mechanism to enhance exploration, while Mask-PPO efficiently masks invalid actions. Next, a refined preference model based on the Bradley–Terry model is employed to resolve overflow issues and establishes a strong negative correlation between predicted rewards and makespan. Moreover, experimental results demonstrate the algorithm's adaptability to large-scale instances, outperforming traditional rule-based and DRL methods. Finally, ablation studies confirm the effectiveness of key components, such as preference computation and policy switching, in significantly enhancing algorithm performance.

In the future work, we plan to address the following tasks: (i) develop an ensemble preference model composed of multiple reward networks, where the mean of their predictions is used to enhance accuracy; (ii) actively select fragment pairs based on uncertainty variance rather than random selection, improving learning efficiency and enhancing model generalization; (iii) extend the proposed algorithm to other combinatorial problem-solving domains, offering significant potential for broader applications.

Acknowledgements This work is supported by the Technology Innovation Fund Project of Dalian Neusoft University of Information (TIFP202307).

Data availability All data that support the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Wang S, Wan J, Li D, Zhang C (2016) Implementing smart factory of Industry 4.0: an outlook. *Int J Distrib Sens Netw* 12:3159805
2. Johnson SM (1954) Optimal two-and three-stage production schedules with setup times included. *Naval Res Logist Q* 1(1):61–68
3. Garey MR, Sethi JR (1976) The complexity of flow-shop and job-shop scheduling. *Math Oper Res* 1(2):117–129
4. Brucker P, Schlie R (1990) Job-shop scheduling with multi-purpose machines. *Computing* 45:369–375
5. Garey MR, Johnson DS, Sethi R (1976) The complexity of flow-shop and jobshop scheduling. *Math Oper Res* 1:117–129
6. Panwalkar SS, Iskander W (1977) A survey of scheduling rules. *Oper Res* 25(1):45–61
7. Pezzella F, Morganti G, Ciaschetti G (2008) A genetic algorithm for the flexible job-shop scheduling problem. *Comput Oper Res* 35(10):3202–3212
8. Sutton RS, Barto AG (2018) Reinforcement learning: an introduction. MIT Press, Cambridge
9. Calleja G, Pastor R (2014) A dispatching algorithm for flexible job-shop scheduling with transfer batches: an industrial application. *Prod Plan Control* 25(2):93–109

10. Ortíz MA, Betancourt LE, Negrete KP et al (2018) Dispatching algorithm for production programming of flexible job-shop systems in the smart factory industry. *Ann Oper Res* 264:409–433
11. Saidi-Mehrabadi M, Fattahi P (2007) Flexible job shop scheduling with tabu search algorithms. *Int J Adv Manuf Technol* 32:563–570
12. Nouiri M, Bekrar A, Jemai A et al (2018) An effective and distributed particle swarm optimization algorithm for flexible job-shop scheduling problem. *J Intell Manuf* 29:603–615
13. Braune R, Benda F, Doerner KF et al (2022) A genetic programming learning approach to generate dispatching rules for flexible shop scheduling problems. *Int J Prod Econ* 243:108342
14. Zhang W, Dietterich TG (1995) A reinforcement learning approach to job-shop scheduling. In: *IJCAI*, vol 95, pp 1114–1120
15. Xue T, Zeng P, Yu H (2018) A reinforcement learning method for multi-AGV scheduling in manufacturing. In: 2018 IEEE international conference on industrial technology (ICIT). IEEE, Lyon, pp 1557–1561
16. Shi D, Fan W, Xiao Y et al (2020) Intelligent scheduling of discrete automated production line via deep reinforcement learning. *Int J Prod Res* 58(11):3362–3380
17. Han BA, Yang J (2021) A deep reinforcement learning based solution for flexible job shop scheduling problem. *Int J Simul Model* 20(2):375–386
18. Oh SH, Cho YI, Woo JH (2022) Distributional reinforcement learning with the independent learners for flexible job shop scheduling problem with high variability. *J Comput Design Eng* 9(4):1157–1174
19. Zhang JD, He Z, Chan WH et al (2023) DeepMAG: deep reinforcement learning with multi-agent graphs for flexible job shop scheduling. *Knowl-Based Syst* 259:110083
20. Du Y, Li J, Chen X et al (2022) Knowledge-based reinforcement learning and estimation of distribution algorithm for flexible job shop scheduling problem. *IEEE Trans Emerg Top Comput Intell* 7(4):1036–1050
21. Bonetta G, Zago D, Cancelliere R et al (2023) Job shop scheduling via deep reinforcement learning: a sequence to sequence approach. In: *International conference on learning and intelligent optimization*. Springer International Publishing, Cham, pp 475–490
22. Li R, Gong WY, Wang L, Lu C, Dong CX (2024) Co-evolution with deep reinforcement learning for energy-aware distributed heterogeneous flexible job shop scheduling. *IEEE Trans Syst Man Cybern Syst* 54(1):201–211
23. Ren T, Dong Z, Qi F et al (2024) Solving the flow-shop scheduling problem with human factors and two competing agents with deep reinforcement learning. *Eng Optim* 1–17
24. Yuan E, Wang L, Cheng S, Song S, Fan W, Li Y (2024) Solving flexible job shop scheduling problems via deep reinforcement learning. *Expert Syst Appl* 245:123019
25. Xu S, Li Y, Li Q (2024) A deep reinforcement learning method based on a transformer model for the flexible job shop scheduling problem. *Electronics* 13:3696. <https://doi.org/10.3390/electronics13183696>
26. Luo S, Zhang L, Fan Y (2021) Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning. *IEEE Trans Autom Sci Eng* 19(4):3020–3038
27. Lei K, Guo P, Zhao W et al (2022) A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem. *Expert Syst Appl* 205:117796
28. Zhao L, Fan J, Zhang C et al (2023) A DRL-based reactive scheduling policy for flexible job shops with random job arrivals. *IEEE Trans Autom Sci Eng* 2023:1–12
29. Chang YH, Liu CH, You SD (2024) Scheduling for the flexible job-shop problem with a dynamic number of machines using deep reinforcement learning. *Information* 15(2):82
30. Hammami NEH, Lardeux B, Hadj-Alouane AB et al (2024) Design and calibration of a DRL algorithm for solving the job shop scheduling problem under unexpected job arrivals. *Flex Serv Manuf J* 1–32
31. Heger J, Voss T (2023) Dynamically adjusting the k-values of the ATCS rule in a flexible flow shop scenario with reinforcement learning. *Int J Prod Res* 61(1):147–161
32. Long X, Zhang J, Qi X et al (2022) A self-learning artificial bee colony algorithm based on reinforcement learning for a flexible job-shop scheduling problem. *Concurr Comput Pract Exp* 34(4):e6658
33. Song W, Chen X, Li Q et al (2022) Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Trans Ind Inf* 19(2):1600–1610
34. Du Y, Li J, Li C et al (2022) A reinforcement learning approach for flexible job shop scheduling problem with crane transportation and setup times. *IEEE Trans Neural Netw Learn Syst* 35(4):5695–5709
35. Yuan M, Huang H, Li Z et al (2023) A multi-agent double deep-Q-network based on state machine and event stream for flexible job shop scheduling problem. *Adv Eng Inform* 58:102230
36. Yang S, Wang J, Xu Z (2024) Learning to schedule dynamic distributed reconfigurable workshops using expected deep Q-network. *Adv Eng Inform* 59:102307
37. Jing X, Yao X, Liu M et al (2024) Multi-agent reinforcement learning based on graph convolutional network for flexible job shop scheduling. *J Intell Manuf* 35(1):75–93
38. Huang D, Zhao H, Zhang L, Chen K (2024) Learning to dispatch for flexible job shop scheduling based on deep reinforcement learning via graph gated channel transformation. *IEEE Access* 12:50935–50948
39. Tang H, Dong J (2024) Solving flexible job-shop scheduling problem with heterogeneous graph neural network based on relation and deep reinforcement learning. *Machines* 12:584. <https://doi.org/10.3390/machines12080584>
40. Zhu Q, Gao K, Huang W et al (2024) Q-learning-assisted meta-heuristics for scheduling distributed hybrid flow shop problems. *Comput Mater Continua* 80(3):3573–3589
41. Huang S, Ontañón S (2020) A closer look at invalid action masking in policy gradient algorithms. *arXiv preprint* <http://arxiv.org/abs/2006.14171>
42. Lawrence S (1984) An experimental investigation of heuristic scheduling techniques. Supplement to resource constrained project scheduling, 1984
43. Akiba T, Sano S, Yanase T et al (2019) Optuna: a next-generation hyperparameter optimization framework. In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp 2623–2631
44. Fisher H, Thompson G Probabilistic learning combinations of local job-shop scheduling rules. In: Muth JF, Thompson GL (eds) *Industrial scheduling*, vol 1963. Prentice-Hall, Upper Saddle River, pp 225–251
45. Demirkol E, Mehta S, Uzsoy R (1998) Benchmarks for shop scheduling problems. *Eur J Oper Res* 109(1):137–141. [https://doi.org/10.1016/s0377-2217\(97\)00019-2](https://doi.org/10.1016/s0377-2217(97)00019-2)
46. Storer RH, Wu SD, Vaccari R (1992) New search spaces for sequencing problems with application to job shop scheduling. *Manag Sci* 38(10):1495–1509. <https://doi.org/10.1287/mnsc.38.10.1495>
47. Taillard E (1993) Benchmarks for basic scheduling problems. *Eur J Oper Res* 64(2):278–285